

A Divisão do Código

Zona 1: O Mapa de Hardware (Pinos e Conexões)

Aqui vamos declarar onde cada fio físico está conectado. Não usaremos números soltos espalhados pelo código. Criaremos "apelidos" (usando `#define`) para que qualquer pessoa saiba que o pino 15 é a direção do motor direito, por exemplo. O compilador substituirá esses nomes pelos números exatos antes de rodar, otimizando o uso da memória.

- **parte 2: A Física do PWM (O Acelerador)**
 - O ESP32 precisa de parâmetros elétricos para gerar o sinal de velocidade (frequência do pulso, resolução em bits e canais de hardware). Essa zona guardará essas configurações.
- **parte 3: As Funções de Movimento (Os Músculos)**
 - Aqui entra o coração do nosso módulo. Em vez de acionar pinos diretamente, vamos criar blocos modulares elegantes e fáceis de ler, como `andar_frente(velocidade)` e `parar_motores()`.
- **parte 4: A Função `setup()` (A Ignição)**
 - O código que o robô roda apenas uma única vez quando você liga a bateria. Ele vai configurar os pinos como "Saídas de Energia" e atrelar os canais do PWM à Ponte H.
- **parte 5: A Função `loop()` (O Teste Prático)**
 - O ciclo infinito. Por enquanto, criaremos uma rotina de teste simples apenas para validar o robô acelerando gradativamente no ar. (Mais para a frente, é nesta zona que colocaremos o servidor web para escutar as ordens do Python).

Parte 1: Mapeamento de Hardware com `#define`

"Para mapear os pinos do ESP32 que estão conectados na Ponte H, utilizamos a diretiva `#define`.

Decidimos seguir esse padrão porque ele otimiza o uso da placa. O `#define` não cria uma variável na memória do robô durante a execução. Ele funciona como um comando de 'Localizar e Substituir' no texto do código.

Antes mesmo de o código ser traduzido para linguagem de máquina (compilação), o sistema procura todas as palavras como `PINO_VEL_ESQ` e troca pelo número 2. Isso nos permite escrever um código fácil de ler para os humanos (usando nomes claros em vez de números soltos), sem gastar nenhuma memória extra do microcontrolador."

parte 2:

Para o robô não dar 'trancos' e ir buscar o balão de forma suave, nós usamos o PWM. Como o ESP32 não consegue mandar 'meia energia' para o motor, o PWM basicamente liga e desliga a energia muito rápido para simular uma velocidade menor. Na **Zona 2** do código, definimos as três regras para isso funcionar:

- **Frequência (1000 Hz):** Vimos que 1000 piscadas por segundo é um valor padrão bom para esses motores amarelos do robô. Isso evita que o motor fique fazendo aquele barulho agudo chato quando anda devagar.
- **Resolução (8 bits):** Isso é só o tamanho do nosso 'acelerador'. Com 8 bits, nós temos 256 opções de velocidade. Então no nosso código, quando quisermos parar a gente manda 0, e quando quisermos força total a gente manda 255.
- **Canais (0 e 1):** A placa ESP32 tem vias separadas para mandar esse sinal. Nós separamos o canal 0 para controlar a velocidade das rodas esquerdas e o canal 1 para as rodas direitas. Precisamos disso separado para poder fazer o robô virar."